



Construcción de software y toma de decisiones (Gpo 501)

**Evidencia Definición de requerimientos de
Software**

Carlos Arturo Gómez Ayala - A01711027
ITESM Campus Querétaro
Profesor: Enrique Alfonso Calderón Balderas
Co-Titular: Alejandro Fernández Vilchis
Co-Titular: Denisse Lizbeth Maldonado Flores

Fecha: 8/06/2026

Requerimientos de Software

Requerimiento: Necesidad, condición o fx que el sistema debe cumplir para resolver o satisfacer Funcionalidades.

Requerimientos Funcionales: Son declaraciones de lo que hará el sistema.

Ejemplos: Registrar usuarios, gestionar catálogos, buscar por nombre, título y mostrar resultados. Se habla de actores, acciones usuario, datos.

Requerimientos No Funcionales: Describen aspectos de calidad del sistema, rendimiento, seguridad, usabilidad, cómo debe comportarse, con qué calidad. Fiabilidad, respuesta en tiempo, capacidad almacenamiento. Define restricciones como capacidad de los dispositivos entrada/salida.

Ejemplos: Requerimientos productivos: rapidez sistema y almacén. Requerimientos organizacionales: políticas y procedimientos de la organización, estándares. Requerimientos externos: leyes, sistemas operativos.

Ejemplos: Seguridad y autenticación: Recuperar contraseña por correo. Escalabilidad: +1,000,000 registros y 10k usuarios. Usabilidad: amigable intuitivo para facilitar uso.

sin comprometer rendimiento.

Disponibilidad: al menos 99% del tiempo.

Mantenibilidad: updates.

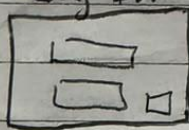
Casos de Uso o Funcionalidades

- Es una descripción de una interacción entre un actor y el sistema para lograr un objetivo.

Responder * ¿Qué quiere hacer el usuario con el sistema?

Ej: Actor	Caso Uso	Objetivo
Usuario A	Reservar mesa	Apartar mesa
	Hacer reservación	

Log In



Campos en todos los posibles casos
No describirlo bien lleva errores.

Casos de Uso entonces es documentar RF de manera gráfica, bajar la ambigüedad.

- Son ante todo **RF**



Actor



Caso
Uso

- Include
← extend

- Representa el comportamiento sistema desde perspectiva de actor.

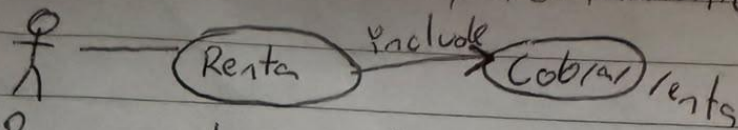
- Describe lo que hace, **No** cómo lo hace.

- Diagrama actor A, funcionalidades o casos O, relaciones.

- El valor es gráfico y ser específico.

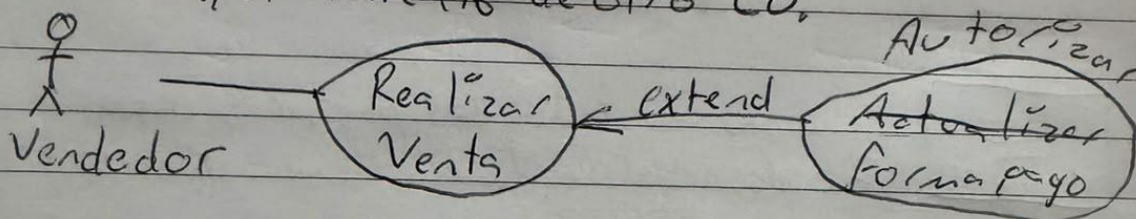
Actor: Quién interactúa con el sistema. Es estímulo al sistema con algún evento o recibe info. del sistema.
El actor es externo al sistema.

Relación include: Esta relación entre casos incluye el caso base e incorpora el comportamiento de otro. Explícitamente



Representa una dependencia y declara que un CU utilice info y servicios de otro CU.

Relación extend: CU incorpora implícitamente el comportamiento de otro CU.



Escenario o Secuencia específica de acciones entre actores y el sistema.

Unified Modelling Language

Caso Uso

Campo

Qué va ahí

ID y nombre

Actor principal

Actor secundario

Descripción

Pre y Postcondiciones

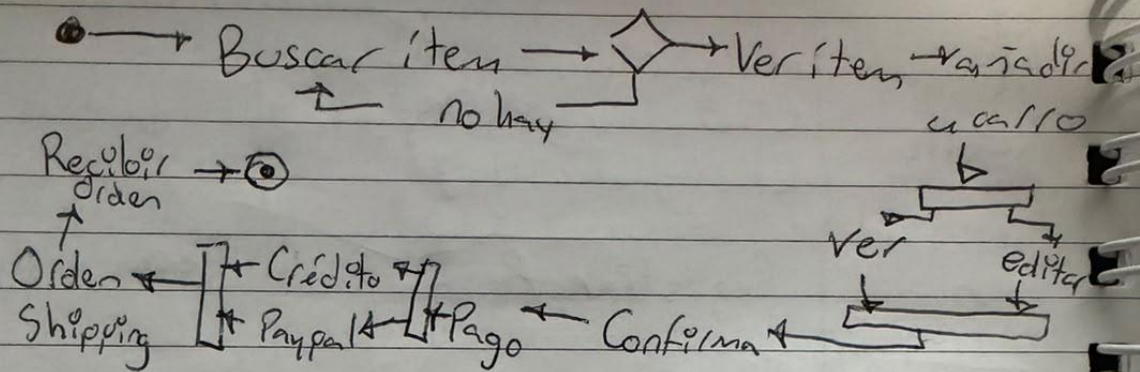
Cumplir antes/después

Paso 1: Pasos acción de CO.

↳ Identificar actividades y acciones

Paso 2: Actores

Paso 3: Flujo actividades



UML - Class Diagram

- Es el más técnico. Muestra qué clases existen, qué datos tienen, qué métodos y relaciones.

Persona

- id: int
- nombre: str
- email: str
+ autenticar

Paciente

- expediente
- fecha Nac
-

+ agendar
+ cancelar
+ reagendar
+ historial

Médico

- cédula
- horario
+ consultar

+ agendar
+ registrar

Requerimientos de software qué son y cómo los clasifiqué

En clase definimos que un requerimiento es una necesidad, condición o función que el sistema debe cumplir para resolver o satisfacer algo. No es solo una lista de deseos, es lo que le da forma al sistema antes de escribir una sola línea de código.

Lo primero que aprendí fue a distinguir los dos grandes tipos:

Requerimientos Funcionales

Son declaraciones de lo que el sistema hará. Se habla de actores, acciones, datos. En CoreData, cada RF viene de una conversación real con SVA LAW, no lo inventamos nosotros. Algunos ejemplos directos del proyecto:

- RF-06 Registrar operación — el analista captura movimientos del cliente como monto, tipo, producto, moneda.
- RF-09 Generar alertas automáticas donde el sistema evalúa cada operación contra reglas activas y genera una alerta sin que nadie se lo pida. Este fue el más crítico de definir bien.
- RF-10 Registrar reporte interno (buzón anónimo) el cliente lo llamó literalmente 'el chimógrafo'. Eso me hizo entender que el vocabulario del interesado importa si él dice chimógrafo, yo tengo que saber a qué RF corresponde.

Lo que me cambió aquí fue entender que los RF no se escriben solos. Cada uno tiene un actor detrás, una acción concreta y un dato que se mueve. Si un RF no puede contestar '¿quién hace qué con qué?', está mal redactado.

Requerimientos No Funcionales

Describen los aspectos de calidad del sistema: rendimiento, seguridad, usabilidad, cómo debe comportarse, con qué nivel de calidad. También definen restricciones como la capacidad de los dispositivos o los canales de entrada/salida.

En clase los vimos clasificados en tres grupos que me resultaron muy útiles para CoreData:

- Requerimientos de producto: rapidez del sistema y almacenamiento. En nuestro caso, las consultas a Supabase tienen que ser suficientemente rápidas para no bloquear al oficial mientras revisa alertas en tiempo real.

- Requerimientos organizacionales, las políticas y procedimientos de la organización, estándares. Para SVA LAW esto es directo de la CNBV que impone los estándares de reporte que el sistema tiene que cumplir sí o sí.
- Requerimientos externos: leyes, sistemas operativos. La LFPDPPP (Ley Federal de Protección de Datos) nos obligó a guardar credenciales como variables de entorno y no exponerlas en el repositorio.

Ejemplos concretos que documentamos en el proyecto

- Seguridad y autenticación donde el 100% de usuarios requieren login. Viene directo de la regulación, no es opcional.
- Escalabilidad: el sistema debe poder manejar el crecimiento del portafolio de clientes de SVA LAW sin degradar el rendimiento.
- Disponibilidad: el oficial de cumplimiento necesita acceso al sistema en cualquier momento para responder a la CNBV.

Antes del curso yo pensaba que los no funcionales eran 'extras'. En CoreData me di cuenta de que, en un sistema regulado, el RNF de trazabilidad registrar quién hizo qué y cuándo es tan importante como cualquier RF, porque sin él el sistema no pasa una auditoría.

Casos de uso: documentar los RF de forma gráfica

Un caso de uso es una descripción de una interacción entre un actor y el sistema para lograr un objetivo. La pregunta que responde es: ¿qué quiere hacer el usuario con el sistema?

En clase vimos que los casos de uso son ante todo RF documentados de manera gráfica y que su valor está en bajar/quitar la ambigüedad. Un formulario que no está bien descrito con todos los campos posibles y casos de error genera problemas en la implementación.

Elementos del diagrama de casos de uso (UML)

El diagrama UML de casos de uso representa el comportamiento del sistema desde la perspectiva del actor. Describe lo que hace, no cómo lo hace. Los componentes que usamos:

- Actor (figura humana): quién interactúa con el sistema. Es externo al sistema, le envía algún evento o recibe información de él. En CoreData tenemos tres actores principales: Analista, Oficial de Cumplimiento y el Sistema.

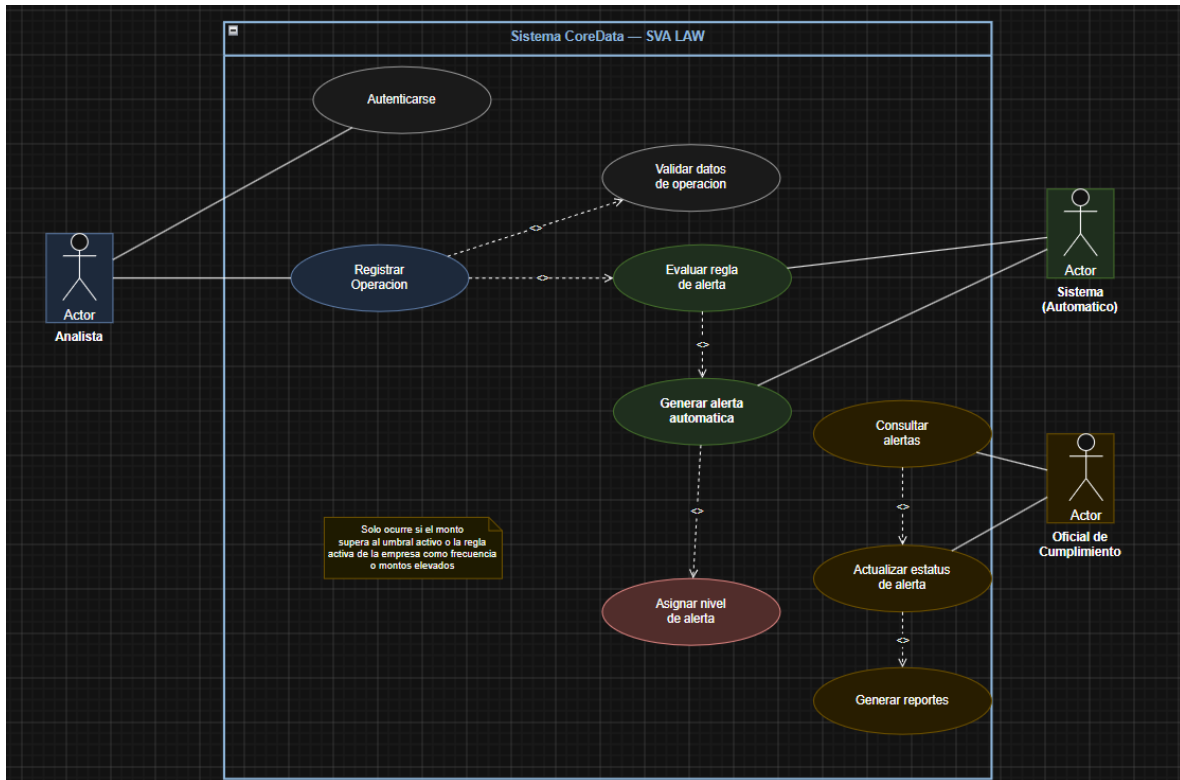
- Caso de uso (óvalo): la funcionalidad o caso. En el diagrama aparece nombrado con un verbo en infinitivo.
- Relación include: cuando un caso base incorpora explícitamente el comportamiento de otro. Representa una dependencia: el CU base declara que utiliza los servicios de otro CU. Por ejemplo, 'Registrar operación' incluye 'Evaluar regla de alerta'.
- Relación extend: cuando un CU incorpora implícitamente el comportamiento de otro bajo ciertas condiciones. No siempre ocurre, solo cuando se cumple la condición. Por ejemplo, 'Generar alerta' extiende 'Registrar operación' solo si se supera el umbral.

La distinción entre include y extend me costó trabajo al principio. Lo que me ayudó fue pensar así fue que un include es obligatorio, siempre pasa. Extend es condicional, solo pasa si algo se cumple. En CoreData la alerta no siempre se genera, depende del monto, por eso es extend.

Diagrama de actividad UML el flujo paso a paso

El UML Activity Diagram es un diagrama de comportamiento que muestra el flujo paso a paso de un caso de uso. Es diferente al de secuencia, que muestra objetos interactuando en el tiempo. El de actividad es más parecido a un flujograma pero con semántica UML precisa.

Registrar Operación y Generar Alerta



Para el proyecto CoreData elaboré el diagrama de actividad del caso de uso 'Registrar operación y generar alerta' que cubre RF06 y RF09.

Reflexión

Esta competencia me costó más de lo que esperaba, no por los conceptos en sí sino por demostrarlos. En Periodo 1, se me dijo que tenía los conceptos pero que solo los dictaba. Cuando leí eso entendí que el problema no era que no supiera la diferencia entre un requerimiento funcional y uno no funcional, sino que nunca lo había conectado con algo concreto que yo hubiera hecho, que no lo pude demostrar al momento de la entrevista y que solo le deje la labor a alguien mas aun teniendo los conocimientos.

aprendí que los conceptos de esta competencia no son independientes entre sí, se relacionan estrechamente con cada una de las competencias de la materia, con algunas más que con otras y por ello la invitación de agendar mas de una entrevista en un mismo horario. Entender bien los tipos de requerimientos te lleva naturalmente a entender los casos de uso y los casos de uso no tienen sentido sin saber quién es el actor y qué necesita realmente, se lleva al diseño de componentes de software y hasta la determinación de patrones. Todo tiene hilo cuando lo ves aplicado, pero si solo los estudias como temas separados se

quedan como definiciones sueltas en el cuaderno. Por ello hay que ir integrando conforme se van haciendo los conocimientos y habilidades.

La lección más importante que me llevo es que documentar requerimientos no es llenar un formato, es el momento donde se decide qué va a hacer el sistema y para quién. Si eso no está claro desde el principio, todo lo que viene después se construye sobre algo que cada uno interpreta diferente.

Referencias

Uso de IA como apoyo en síntesis de evidencia

Tenía dos fuentes de información para esta competencia, un documento Word con notas digitales de clase del primer intento y apuntes escritos a mano en mi cuaderno físico. *Ninguna estaba completa sola ni organizada para cumplir con la competencia de STC0202.* Le compartí ambas cosas a la IA y le pedí lo siguiente: "Tengo mis apuntes de clase en un Word y en fotos de mi cuaderno sobre requerimientos de software casos de uso include/extend y diagramas de actividad UML y algunos mas de UML. Necesito estudiar y presentar entrevista de dominio de esos apuntes por lo que requiero mas enfoque, mayor informacion disponible o que les falta de informacion."